

Arduino LoRa Implementation

Hardware: Common LoRa Modules

SX1276/SX1278 (Semtech):

- 868 MHz (Europe), 915 MHz (US), 433 MHz
- Output power: +20 dBm (100 mW)
- Sensitivity: -148 dBm
- Popular breakouts: Hope RFM95W, Ai-Thinker Ra-02

RFM95W (HopeRF):

- Based on SX1276
- SPI interface
- 3.3V logic
- Cheap (~\$5-15)

Pins (SPI):

- 3.3V → VCC
- GND → GND
- SCK → Arduino SCK (13)
- MISO → Arduino MISO (12)
- MOSI → Arduino MOSI (11)
- NSS → Arduino D10 (CS)
- DIO0 → Arduino D2 (IRQ)
- RST → Arduino D9 (Reset)

Arduino Code

Basic LoRa Transmitter

```
#include <SPI.h>
#include <LoRa.h>

// Pin definitions
#define SS 10
#define RST 9
#define DI00 2

// LoRa frequency (change based on region)
#define BAND 915E6 // 915 MHz (US)
// Europe: 868E6, Asia: 433E6

int counter = 0;

void setup() {
  Serial.begin(9600);
  while (!Serial);

  Serial.println("LoRa Transmitter");

  // Configure LoRa pins
  LoRa.setPins(SS, RST, DI00);

  // Initialize LoRa
  if (!LoRa.begin(BAND)) {
    Serial.println("Starting LoRa failed!");
    while (1);
  }

  // Set LoRa parameters
  LoRa.setSpreadingFactor(7); // SF7 (fastest)
  LoRa.setSignalBandwidth(125E3); // 125 kHz
```

```
void loop() {  
  Serial.print("Sending packet: ");  
  Serial.println(counter);  
  
  // Send packet  
  LoRa.beginPacket();  
  LoRa.print("Hello ");  
  LoRa.print(counter);  
  LoRa.endPacket();  
  
  counter++;  
  
  delay(5000); // Send every 5 seconds  
}
```

Arduino Code: Basic LoRa Receiver

```
#include <SPI.h>
#include <LoRa.h>

#define SS 10
#define RST 9
#define DI00 2
#define BAND 915E6

void setup() {
  Serial.begin(9600);
  while (!Serial);

  Serial.println("LoRa Receiver");

  LoRa.setPins(SS, RST, DI00);

  if (!LoRa.begin(BAND)) {
    Serial.println("Starting LoRa failed!");
    while (1);
  }

  // Must match transmitter settings!
  LoRa.setSpreadingFactor(7);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(5);

  Serial.println("LoRa Receiver Ready");
}
```

```
void loop() {  
  // Try to parse packet  
  int packetSize = LoRa.parsePacket();  
  
  if (packetSize) {  
    // Received a packet  
    Serial.print("Received packet: ");  
  
    // Read packet  
    while (LoRa.available()) {  
      Serial.print((char)LoRa.read());  
    }  
  
    // Print RSSI (signal strength)  
    Serial.print("' with RSSI ");  
    Serial.println(LoRa.packetRssi());  
  
    // Print SNR (signal-to-noise ratio)  
    Serial.print("SNR: ");  
    Serial.println(LoRa.packetSnr());  
  }  
}
```

Arduino: Temperature Sensor with LoRa

Transmitter (Field Sensor)

```
#include <SPI.h>
#include <LoRa.h>

#define SS 10
#define RST 9
#define DI00 2
#define BAND 915E6
#define TEMP_PIN A0
```

```
void setup() {  
    Serial.begin(9600);  
    LoRa.setPins(SS, RST, DI00);  
  
    if (!LoRa.begin(BAND)) {  
        Serial.println("LoRa init failed!");  
        while (1);  
    }  
  
    LoRa.setSpreadingFactor(12); // SF12 for max range  
    LoRa.setSignalBandwidth(125E3);  
  
    Serial.println("Temperature Sensor Ready");  
}
```

```

void loop() {
  // Read temperature (LM35: 10mV per °C)
  int sensorValue = analogRead(TEMP_PIN);
  float voltage = sensorValue * (5.0 / 1023.0);
  float temperature = voltage * 100.0;

  // Create JSON packet
  String packet = "{\"temp\": ";
  packet += String(temperature, 1);
  packet += ", \"id\": 1}";

  // Send via LoRa
  LoRa.beginPacket();
  LoRa.print(packet);
  LoRa.endPacket();

  Serial.print("Sent: ");
  Serial.println(packet);

  // Sleep for 10 minutes (600 seconds)
  // In real deployment, use low-power sleep mode
  delay(600000);
}

```

Arduino: LoRa Gateway to WiFi

Receiver + WiFi Bridge

```
#include <SPI.h>
#include <LoRa.h>
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>

#define SS 15          // D8 on ESP8266
#define RST 16         // D0
#define DI00 5         // D1
#define BAND 915E6

const char* ssid = "YourWiFi";
const char* password = "YourPassword";
const char* serverUrl = "http://yourserver.com/api/sensor":
```

```
void setup() {  
  Serial.begin(115200);  
  
  // Connect to WiFi  
  WiFi.begin(ssid, password);  
  while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
  }  
  Serial.println("\nWiFi connected");  
  
  // Initialize LoRa  
  LoRa.setPins(SS, RST, DI00);  
  if (!LoRa.begin(BAND)) {  
    Serial.println("LoRa init failed!");  
    while (1);  
  }  
  
  LoRa.setSpreadingFactor(12);  
  Serial.println("LoRa Gateway Ready");  
}
```

```

void loop() {
  int packetSize = LoRa.parsePacket();

  if (packetSize) {
    String data = "";

    while (LoRa.available()) {
      data += (char)LoRa.read();
    }

    Serial.print("Received: ");
    Serial.println(data);

    // Forward to internet server
    WiFiClient client;
    HTTPClient http;

    http.begin(client, serverUrl);
    http.addHeader("Content-Type", "application/json");

    int httpCode = http.POST(data);

    if (httpCode > 0) {
      Serial.print("HTTP Response: ");
      Serial.println(httpCode);
    }

    http.end();
  }
}

```

Common LoRa Problems & Solutions

Problem 1: No Communication / Can't Receive

Symptoms:

```
LoRa.parsePacket() always returns 0
```

Possible causes:

1. Wrong frequency

```
// US: 915 MHz, Europe: 868 MHz, Asia: 433 MHz  
LoRa.begin(915E6); // Check your region!
```

2. Mismatched parameters

```
// TX and RX must match:  
// - Spreading Factor  
// - Bandwidth  
// - Coding Rate  
// - Frequency
```

3. Antenna not connected

- x Never transmit without antenna!
- Can damage module
- Use proper 868/915 MHz antenna
- Check antenna connector

4. Out of range

Start with devices 1 meter apart
Verify communication works
Then increase distance

Problem 2: Short Range

Expected vs Actual:

Expected: 10 km

Actual: 500 m

Why?

Solutions:

1. Increase Spreading Factor

```
LoRa.setSpreadingFactor(12); // Longest range  
// Trade-off: Slower speed, more power
```

2. Increase TX Power

3. Proper Antenna

- ✓ Use outdoor antenna (5 dBi gain)
- ✓ Mount as high as possible
- ✓ Line of sight is best
- ✓ Avoid obstacles (buildings, trees)

4. Check RSSI/SNR

```
Serial.print("RSSI: ");  
Serial.println(LoRa.packetRssi());  
Serial.print("SNR: ");  
Serial.println(LoRa.packetSnr());
```

```
// Good signal: RSSI > -120 dBm, SNR > 0 dB  
// Weak signal: RSSI < -130 dBm, SNR < -10 dB
```

Problem 3: High Packet Loss

Symptoms:

- Only 50% of packets received
- Intermittent communication

Causes & Solutions:

1. Interference

433 MHz band is crowded (remotes, doorbells)
Solution: Use 868/915 MHz if possible

2. Collisions (multiple transmitters)

```
// Add random delay before transmit  
delay(random(100, 5000));  
LoRa.beginPacket();  
// ... send ...
```

3. Moving too fast

Doppler effect at high speeds (>100 km/h)
Solution: Lower spreading factor

4. Weak signal

Check RSSI, improve antenna placement

Problem 4: Module Damage

Symptoms:

- Module gets hot
- No communication at all
- Burnt smell

Prevention:

Don't do this:

- Never transmit without antenna
- Never reverse power polarity

Always do this:

- Always use 3.3V (not 5V!)
- Add antenna BEFORE powering on
- Use proper RF connectors